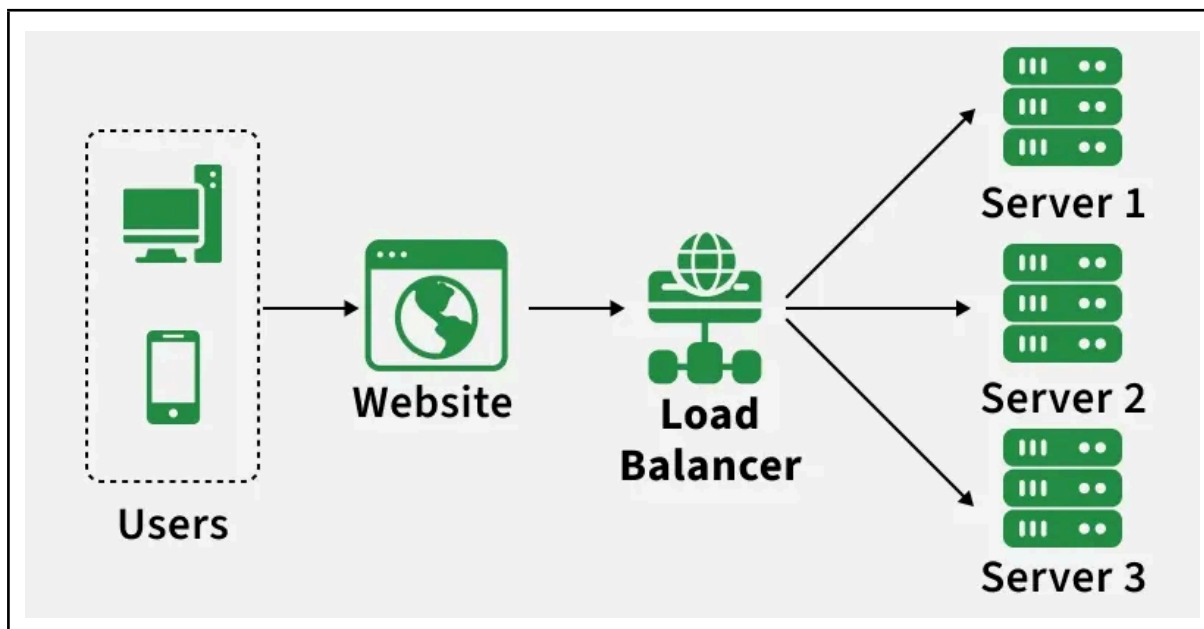


Section 2: System Design Fundamentals

Introduction To Load Balancing

In this session, we will explore why load balancing is essential in system architecture, the different types of load balancers, and the various strategies used to distribute traffic effectively. Load balancing plays a critical role in ensuring high availability, scalability, and reliability in modern applications.

By the end of this session, you will have a clear understanding of how load balancers work and how to choose the right approach for your system.



Why we need Load Balancers?

In modern systems, handling traffic efficiently is crucial to ensuring smooth user experiences. This is where the load balancers come into picture. The below are the key reasons why load balancers are essential:

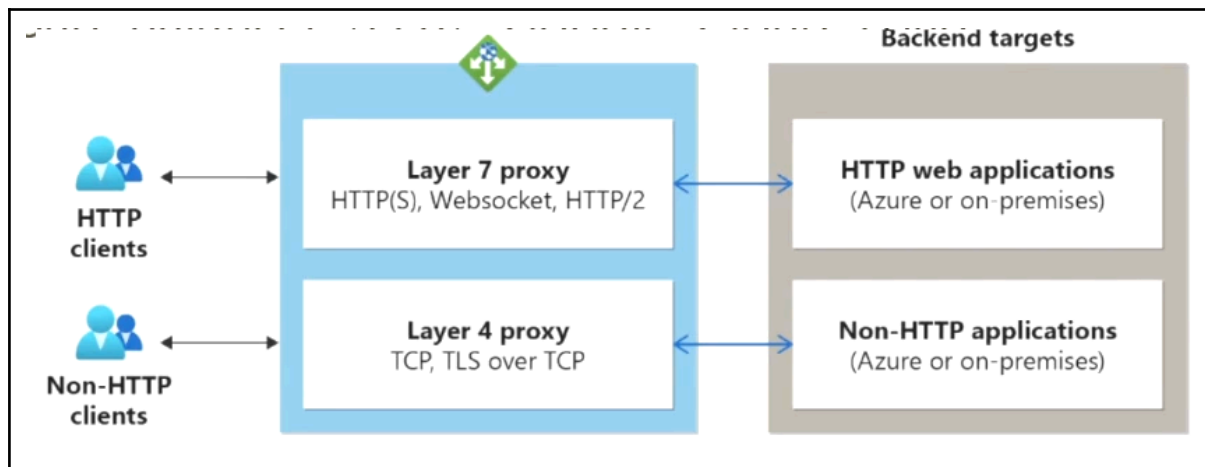
- **High Availability:** A load balancer ensures that our system stays up and running even when the traffic spikes. If one server is down, the traffic is automatically redirected to the healthy servers, which will minimize the downtime for the application.
- **Traffic Distribution:** Instead of overwhelming a single server, a load balancer spreads out the request evenly across multiple servers. This ensures no one machine bears too much load.
- **Prevents Overload:** Without load balancing, a sudden surge in traffic could crash a server, leading to service disruptions. Load balancers help prevent this by distributing workload intelligently.
- **Improves Performance:** By sending requests to the least busy server or fastest responding server, a load balancer can reduce overall latency and ensure users get a faster response time.
- **Failure Handling:** If a server crashes or becomes unresponsive, the load balancer detects it and reroutes the request to other healthy servers, ensuring the continuous service to the end user.
- **Scalability:** As traffic to our web server goes, as traffic to our website grows, we add more servers and the load balancer will automatically distribute requests among them, allowing our systems to scale efficiently.

To put all of this into perspective, consider a high traffic e-commerce website like Amazon during a sale event. Without load balancing, a sudden influx of users could overwhelm a single server, causing slow performance or even downtime in some cases. But with a load balancer in place, incoming requests are intelligently distributed across multiple servers, and it ensures a smooth shopping experience for all the customers. This is why load balancing is a fundamental concept of modern system design.

Types of Load Balancers

Load Balancers can be categorized based on 2 key aspects:

- **Based on Layer:** The load balancer can be categorized based on the OS layer of networking on which they operate. There are 2 main types of load balancers, layer 7 load balancers and layer 4 load balancers.



- ◆ **Level 4 (Transport Layer):** This load balancer works on the transport layer, meaning they make the routing decision based on the network level data such as IP addresses and the TCP or UDP ports. They do not inspect the actual content of the request because they cannot. Only the metadata surrounding is visible to the layer 4 load balancer due to which these balancers are extremely fast and efficient, and it makes them the ideal choice in scenarios where speed and performance are the highest priority. Examples: AWS network load balancer, HAProxy operate in the layer 4 mode.
- ◆ **Layer 7 (Application Layer):** They operate on the application layer. The layer 7 load balancers inspect the actual request contents also. This allows them to make intelligent routing decisions based on the factors such as what HTTP headers are present, what cookies are there, what query strings are there, what is the request path, and even the user's location. For example, an e-commerce site could use a layer 7 load balancer to redirect traffic for product pages to one set of servers while sending checkout requests to a different set of servers, which are optimized for transactions. This flexibility makes layer 7 load balancers powerful, but it also means that they require more processing power compared to layer 4 load balancers. Examples: AWS application load balancers, Nginx.

→ **Based on Deployment Model:** Apart from the above network layer, load balancers are also classified based on how they are deployed. There are mainly 3 categories for this hardware, software, and cloud-based load balancers.

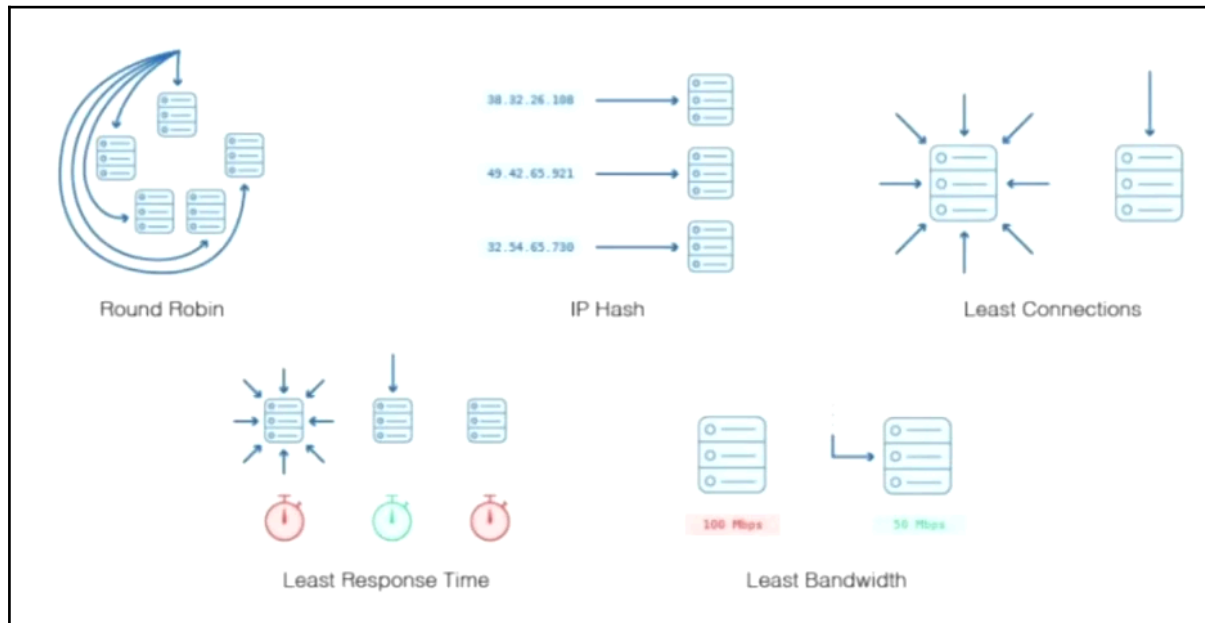
- ◆ **Hardware-based load balancers:** Hardware load balancers are physical devices specifically built for high-performance load balancing. These are used in large enterprises or data centers that require dedicated solutions for handling millions of requests per second. These balancers often come with built-in features like SSL termination, DDoS protection, and advanced protection. Some of the famous hardware load balancers are F5 network, Big IP load balancers, and Citrix NetScaler.
- ◆ **Software load balancers:** The applications that run on general-purpose servers. They provide flexibility and cost-effectiveness because no dedicated hardware is required. They can be deployed on any virtual machine, container, or even on-premises infrastructure. The popular open-source software load balancers include Nginx, HAProxy, and Inverse. They are widely used in modern architecture, including the Kubernetes-based microservices environments.
- ◆ **Cloud-based load balancers:** With the rise of cloud computing, many organizations now use cloud-based load balancers. These are managed services provided by our cloud vendors that automatically distribute traffic across virtual machines, containers, or even multiple geographical regions. These load balancers eliminate the need for manual infrastructure management and provide scalability, security and global traffic distribution out of the box. Some examples are AWS Elastic Load Balancers, Google Cloud Load Balancers, Azure Load Balancers etc.

So, in summary, the load balancer classification can be based on their network layer, layer 4 for the fast transport-level distribution, and layer 7 for the content-aware intelligent routing. Additionally they can be deployed as hardware applications for enterprise needs, software applications for flexible deployment, or as a cloud-based solution for seamless scaling.

Choosing the right type of load balancers depends on the use case. What performance requirements are there and what infrastructure requirements are there?

Load Balancing Strategies

Load balancing strategies can be broadly categorised into 2 categories. The choices of strategy depend on the system architecture, the traffic patterns, and the server capabilities.



- **Static Load Balancing:** This strategy involves distributing traffic based on predefined rules without considering the real-time server conditions. These strategies work very well in environments where the server capacities are similar and the workloads are predictable. The techniques involved in this strategy are:
 - **Round Robin:** It is one of the simplest load balancing techniques. Round robin sends requests sequentially to each server in a circular order. Once the last server receives the request, the process starts over from the first server again. This cycle ensures an even distribution of traffic, but it does not consider the server load or the response time that each server is providing.
 - **Least Connections:** This strategy directs how new requests to the server with fewest active connections. This is specially useful for applications where the request processing times are significantly different from each other. For example, in a database system, some queries might take longer time, some might take less time. This strategy ensures that a heavily loaded server does not receive more requests until it is clearing the existing ones that it has.
 - **IP Hashing:** In IP hashing, a client's IP address determines which server handles the request. A hash function maps each IP address to a specific server, ensuring that requests from the same clients are consistently routed to the same backend server. This is useful in scenarios where we require session persistence such as when a user is logging into a system or a website, and it needs to stay connected to the same server for the duration of

the session. Many load balancers also coin the term as session affinity or sticky session for this kind of technique where they want each user session to go to the same server for that particular session.

Now, the static strategies are simple and effective in certain scenarios, but they are not optimal for the systems with the fluctuating workloads. That's where the dynamic load balancing comes in.

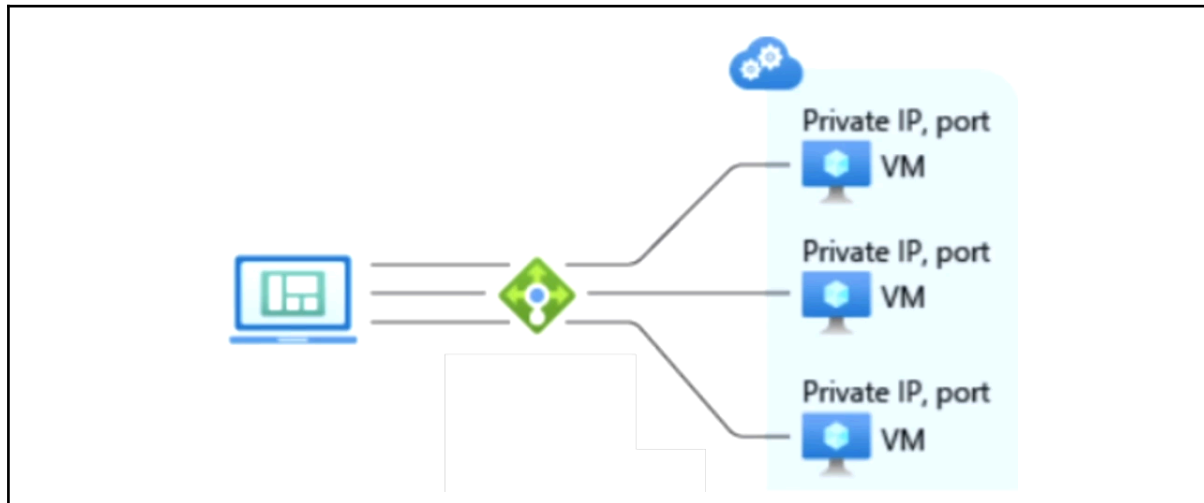
- **Dynamic Load Balancing:** Unlike static strategies, dynamic load balancing takes real time server performance into account when distributing traffic. These strategies continuously monitor server conditions to optimise request routing.
 - **Least Response Time:** This method directs requests to the server with the lowest response time, ensuring that the users get the fastest possible responses. It is particularly useful for applications where speed is very critical, such as financial trading platforms or live streaming video services.
 - **Adaptive Load Balancing:** Adaptive load balancing continuously analyzes the real time data such as CPU usage, memory utilization and server health to make intelligent routing decisions. For instance, if one of the servers is experiencing high CPU usage, the load balancer will dynamically shift traffic to healthier servers, preventing the overload of the already overloaded server and thus improving the overall performance.
 - **Weighted load balancing:** In the weighted load balancing, different servers are assigned weights based on their capacity. For example, a powerful server with high processing power may receive twice as many requests as the less capable server. This server is useful when infrastructure consists of heterogeneous machines with varying performance levels with different system specifications. Some of them are very powerful, some of them are not that powerful.

So all these dynamic load balancing strategies ensure better resource utilization and system reliability by adapting to the traffic conditions in real time. However, they require more monitoring and processing overheads compared to the static load balancing methods.

To summarize, static load balancing methods like round robin, least connection and IP Hashing use predefined rules, making them simple but not adaptive. On the other hand, dynamic strategies like least response time, adaptive load balancing and weighted load balancing provides much smarter traffic and distribution decisions based on the server health and performance. Choosing the right load balancing strategy depends on what your system needs are. If all servers have equal capacity, round robin may work well. But in systems with fluctuating workloads, dynamic strategies like adaptive load balancing are much more effective.

Load Balancer in Action

- Example Scenario:
 - A web application with multiple servers.
 - Users send requests, and the load balancer distributes them efficiently.



Imagine a web application that serves thousands or even millions of users daily. These applications run on multiple backend servers to handle this load efficiently. However, without a load balancer, user requests either overload a single server or require them to manually select a server, which isn't very practical. That's where load balancer comes in.

When the user sends a request, whether it's offloading a web page, making a payment or submitting a form, the load balancer acts as an intermediary. Instead of requests going to a single server, the load balancer distributes them across multiple servers on the chosen strategy.

For example, if we use round robin strategy, the first request is sent to server 1, the next to server 2 and then so on. If we use the least connection strategy, then the server with the fewest active connections will get the request, ensuring the efficient resource utilization.

Now what happens if one of the backend servers crashes? The load balancer continuously monitors the server health. It detects the failures and reroutes the traffic to the remaining healthy servers. This ensures that the user experiences no downtime and we will keep maintaining our high availability and reliability of the application. The real example of this could be a high traffic e-commerce platform like Amazon. During peak shopping hours, millions of users are browsing and placing orders simultaneously. The load balancer ensures that requests are efficiently distributed across multiple servers, preventing slow response times and system crashes.

So this demonstrates how load balancing is crucial for ensuring system reliability, scalability and seamless user experience.

Choosing the Right Load Balancer

The choice of a load balancer depends on multiple factors, including the network layer it operates on, scalability requirements, security considerations and the specific use case of the application.

- **First Decision (Layer 4 vs. Layer 7):**

We use a layer 4 load balancer when we want fast and efficient traffic distribution purely based on network level data like IP addresses and ports. It is ideal for handling TCP and UDP traffic where the content inspection is not required.

On the contrary, we use a layer 7 load balancer when we need more intelligent traffic routing decisions based on their HTTP headers, cookie, request paths or query strings. This is useful in API gateways, microservices or anything that is content aware that requires a content aware distribution.

- **Second Decision (Scalability Needs):**

As our application grows, handling the increasing traffic becomes a challenge. So choosing the right load balancer depends on the expected workload also. So for small to medium scale applications, software based load balancers like Nginx or HAProxy could be a very cost effective solution. But for high traffic applications, cloud based load balancers like AWS, ELB or Google Cloud load balancers, they will automatically scale based on demand. For enterprise level workloads, hardware level load balancers are much better suited as they provide lower latency and they are ideal for the mission critical applications.

In case our application experiences unpredictable spikes in traffic, using a cloud based load balancer will also ensure the seamless scaling without manual intervention because cloud based load balancers come with auto scaling options.

- **Security:**

SSL termination and DDoS protection is something that every system designer should be aware of. Encryption and decryption of HTTP traffic requires significant processing power and many load balancers actually come with this capability. So if you choose a load balancer that provides the capability of SSL termination, offloading this process from the backend server will actually improve the performance of your overall application by a lot. DDoS protection is also important. Cloud based and enterprise grade load balancers come with built in DDoS mitigation features. They detect and filter out malicious traffic before it even reaches the backend servers.

So if your application handles very sensitive data, like you have a banking application, healthcare application, and you need strong encryption and DDoS protection, then you should keep these things in mind before choosing the right load balancer that provides the capability of SSL termination and DDoS protection.

- **Use Cases:**
 - Nginx/HAProxy for web applications.
 - AWS ELB for cloud-native applications.
 - Hardware Load Balancers for your system is crucial.

Summary & Key Takeways

- Load balancing enhances scalability, reliability, and performance.
- Different types exist based on layers and deployment models.
- Choosing the right strategy depends on traffic patterns and system needs/
- Essential for highly available and resilient architectures.
- What's next?
 - ◆ API Gateway

Interview Questions on Load Balancing

- Fundamentals
 - What is load balancing, and why is it important?
 - Explain the difference between layer 4 and layer 7 load balancing.
 - How does a load balancer handle high availability and failover?
- Strategies & Use cases
 - Compare round robin and least connection strategies.
 - What are the advantages of weighted load balancing?
 - When would you use a software load balancer over a hardware one?
- Real-World Scenarios
 - How would you design a scalable solution for a large e-commerce site?
 - What factors should be considered when choosing a load balancing strategy?
 - How does a load balancer improve security?